



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第七章 搜索结构



- 静态搜索结构
- 二叉搜索树
- AVL树



7.1 静态搜索结构



搜索(Search)的概念

- 所谓搜索，就是在数据集合中寻找满足某种条件的数据对象。
- 搜索的结果通常有两种可能：
 - 搜索成功，即找到满足条件的数据对象。作为结果，可报告该对象在结构中的位置，还可给出该对象中的具体信息。
 - 搜索不成功，或搜索失败。作为结果，应报告一些信息，如失败标志、位置等。



- 通常称用于搜索的数据集合为**搜索结构**，它是由同一数据类型的对象(或记录)组成。
- 在每个对象中有若干属性，其中有一个属性，其值可唯一地标识这个对象，称为**关键码**。
- 使用**基于关键码的搜索**，搜索结果应是唯一的。
- 在实际应用时，搜索条件是多方面的，可使用**基于属性的搜索**方法，但搜索结果可能不唯一。



- **实施搜索时有两种不同的环境。**
 - ◆ 静态环境，搜索结构在插入和删除等操作的前后不发生改变。

—— 静态搜索表
 - ◆ 动态环境，为保持较高的搜索效率, 搜索结构在执行插入和删除等操作的前后将自动进行调整，结构可能发生变化。

—— 动态搜索表



7.1.1 静态搜索表



- 静态搜索结构指的是数据集合在搜索过程中不发生变化（即没有插入、删除操作，或这些操作很少），所有数据在搜索前已经确定并组织好。
- 在静态搜索表中，数据元素存放于数组中，利用数组元素的下标作为数据元素的存放地址。
- 搜索算法根据给定值 k ，在数组中进行搜索。直到找到 k 在数组中的存放位置或可确定在数组中找不到 k 为止。



数据表与搜索表的类定义



```
#include <iostream.h>
```

```
#include <assert.h>
```

```
const int defaultSize = 100;
```

```
template <class K, class E> //数据表类的前视定义, K  
    为关键码对应的类, E为其他属性对应的类
```

```
class dataList;
```

```
template <class K, class E >
```

```
class dataNode { //数据表中结点类的定义
```

```
    friend class dataList<K, E>;
```

```
    //声明其友元类为dataList
```



public:

dataNode (**const** K x, const E e) :

key(x), other(e) { }

//构造函数

K getKey() **const** { return key; }

//读取关键码

void setKey (K x) { key = x; }

//修改关键码

private:

K key;

//关键码域

E other;

//其他域 (视问题而定)

};



```
template <class K, class E >
```

```
class dataList { //数据表类定义
```

```
public:
```

```
dataList (int sz = defaultSize) : ArraySize(sz), CurrentSize(0)  
{
```

```
    Element = new dataNode<K, E>[sz];
```

```
    assert (Element != NULL);
```

```
}
```

```
dataList (dataList<K, E>& R); //复制构造函数
```

```
virtual ~dataList() { delete []Element; } //析构函数
```

```
virtual int Length() { return CurrentSize; } //求表的长度
```

```
virtual K getKey (int i) const {
```

```
    //提取第  $i$  个 (从1开始) 元素的key值
```



```
    assert (i > 0 & i <= CurrentSize);  
    return Element[i-1].key;  
}  
virtual void setKey (K x, int i) {  
    //修改第 i 个 (从1开始) 元素的key值  
    assert (i > 0 & i <= CurrentSize);  
    Element[i-1].key = x;  
}  
virtual int SeqSearch (const K x) const;    //搜索  
virtual bool Insert (K x, E e1);           //插入  
virtual bool Remove (K x, E& e1);          //删除  
friend ostream& operator << (ostream& out,  
    const dataList<K, E>& OutList);        //输出
```



```
friend istream& operator >> (istream& in,  
    dataList<K, E>& InList);           //输入  
protected:  
    dataNode<K, E> *Element;           //数据表存储数组  
    int ArraySize, CurrentSize;  
                                           //数组最大长度和当前长度  
};
```



```
template <class K, class E>
bool dataList<K, E>::Insert (K k, E e1) {
//在dataList的尾部插入新元素, 若插入失败, 函数返回false,
//否则返回true.
    if (CurrentSize == ArraySize) return false;
    dataNode<K, E> node = dataNode<K, E> (k,e1);
    Element[CurrentSize] = node;    //插入在尾端
    CurrentSize++;
    return true;
};
```



```
template <class K, class E>
bool dataList<K, E>::Remove (K x, E& e1) {
//在dataList中删除关键码为x的元素, 通过e1返回。
//用尾元素填补被删除元素。
    if (CurrentSize == 0) return false;
    for (int i == 0; i < CurrentSize &&
        Element[i].key != x; i++)           //在表中顺序寻找
        if (i == CurrentSize) return false; //未找到
    e1 = Element[i].other;                   //找到, 保存被删元素的值
    Element[i] = Element[CurrentSize-1];    //填补
    CurrentSize--;
    return true;
};
```



7.1.2 顺序搜索 (Sequential Search)



- 顺序搜索主要用于在线性表中搜索。
 - 基本思想：从表的一端开始，逐个比较每个元素，直到找到目标或遍历完整个表。
 - 数据结构：数组或链表
- 设若表中有 `CurrentSize` 个元素，则顺序搜索从表的最前端开始，顺序用各元素的关键码与给定值 x 进行比较。
- 若找到与其值相等的元素，则搜索成功，给出该元素在表中的位置。
- 若整个表都已检测完仍未找到关键码与 x 相等的元素，则搜索失败，给出失败信息。



- 一般的顺序搜索算法在第二章已经讨论过,本章介绍一种使用 “**监视哨**” 的顺序搜索方法。
- 设在数据表 `dataList` 中顺序搜索关键码与给定值 x 相等的数据元素, 要求数据元素在表中从下标 0 开始存放, 下标为 `CurrentSize` 的元素作为控制搜索过程自动结束的 “监视哨” 使用。
 - 若搜索成功, 则函数返回该元素在表中序号 `Location` (比下标大 1) ;
 - 若搜索失败, 则函数返回 `CurrentSize+1`。



使用监视哨的顺序搜索算法



```
template <class K, class E>
int dataList<K, E>::SeqSearch (const K x) const {
    Element[CurrentSize].key = x; //将x设置为监视哨
    int i = 0;
    while (Element[i].key != x) i++;
    //从前向后顺序搜索
    return i+1;
};
```




顺序搜索的递归算法



- 采用递归方法搜索值为 x 的元素，每递归一层就向待查元素逼近一个位置，直到到达该元素。假设待查元素在第 i ($1 \leq i \leq n$) 个位置，则算法递归深度达 i ($1 \sim i$)。

搜索 **30**

$i = 1$

10	20	30	40	50	60
----	----	----	----	----	----

$i = 2$

10	20	30	40	50	60
----	----	----	----	----	----

$i = 3$

10	20	30	40	50	60
----	----	----	----	----	----

递归



顺序搜索的递归算法



```
template <class K, class E>
```

```
int dataList<K, E>::
```

```
SeqSearch (const K x, int loc) const {
```

```
//在数据表 Element[1..n] 中搜索其关键码与给定值  
//匹配的对象, 函数返回其表中位置。参数 loc 是在  
//表中开始搜索的位置
```

```
    if (loc > CurrentSize) return 0;           //搜索失败
```

```
    else if (Element[loc-1].key == x) return loc;
```

```
                                                //搜索成功
```

```
    else return SeqSearch (x, loc+1);          //递归搜索
```

```
};
```



顺序搜索的平均搜索长度



- 设数据表中有 n 个元素，搜索第 i 个元素的概率为 p_i ，搜索到第 i 个元素所需比较次数为 c_i ，则搜索成功的平均搜索长度：

$$ASL_{succ} = \sum_{i=1}^n p_i \cdot c_i \quad \left(\sum_{i=1}^n p_i = 1 \right)$$

- 在顺序搜索并设置 “**监视哨**” 情形：

$c_i = i$, $i = 1, \dots, n$, 因此

$$ASL_{succ} = \sum_{i=1}^n p_i \cdot i$$



- 一般表中各个元素的搜索概率不同，如果按搜索概率的高低排列表中的元素，从顺序表的情况可知，能够得到好的平均搜索长度。
- 在等概率情形， $p_i = 1/n, i = 1, 2, \dots, n$ 。搜索成功的平均搜索程度为：

$$ASL_{succ} = \sum_{i=1}^n \frac{1}{n} \cdot i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}.$$

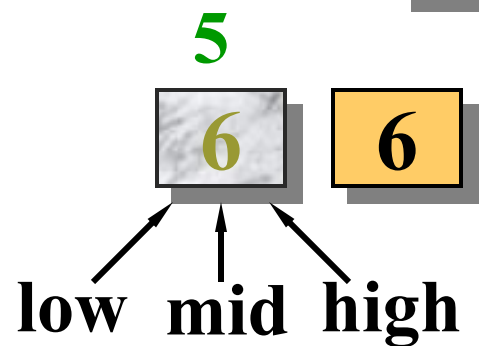
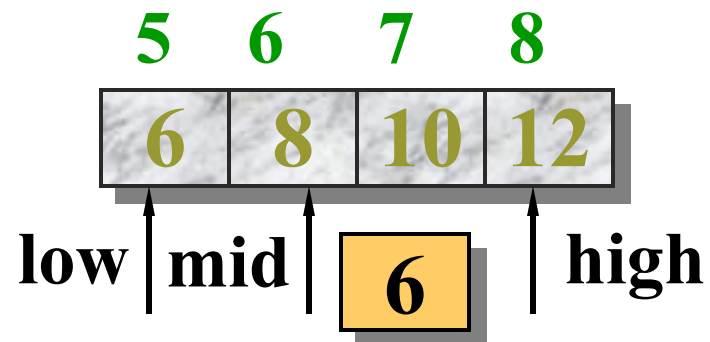
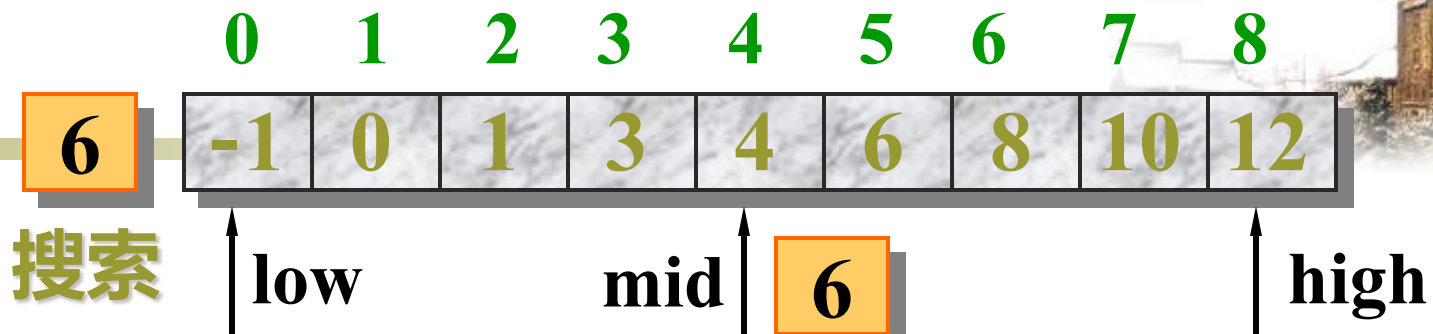
- 搜索不成功的 $ASL_{unsucc} = n+1$ 。
- 优点：实现简单，对存储结构无要求（顺序/链式均可）
- 缺点：效率低， n 较大时不宜使用



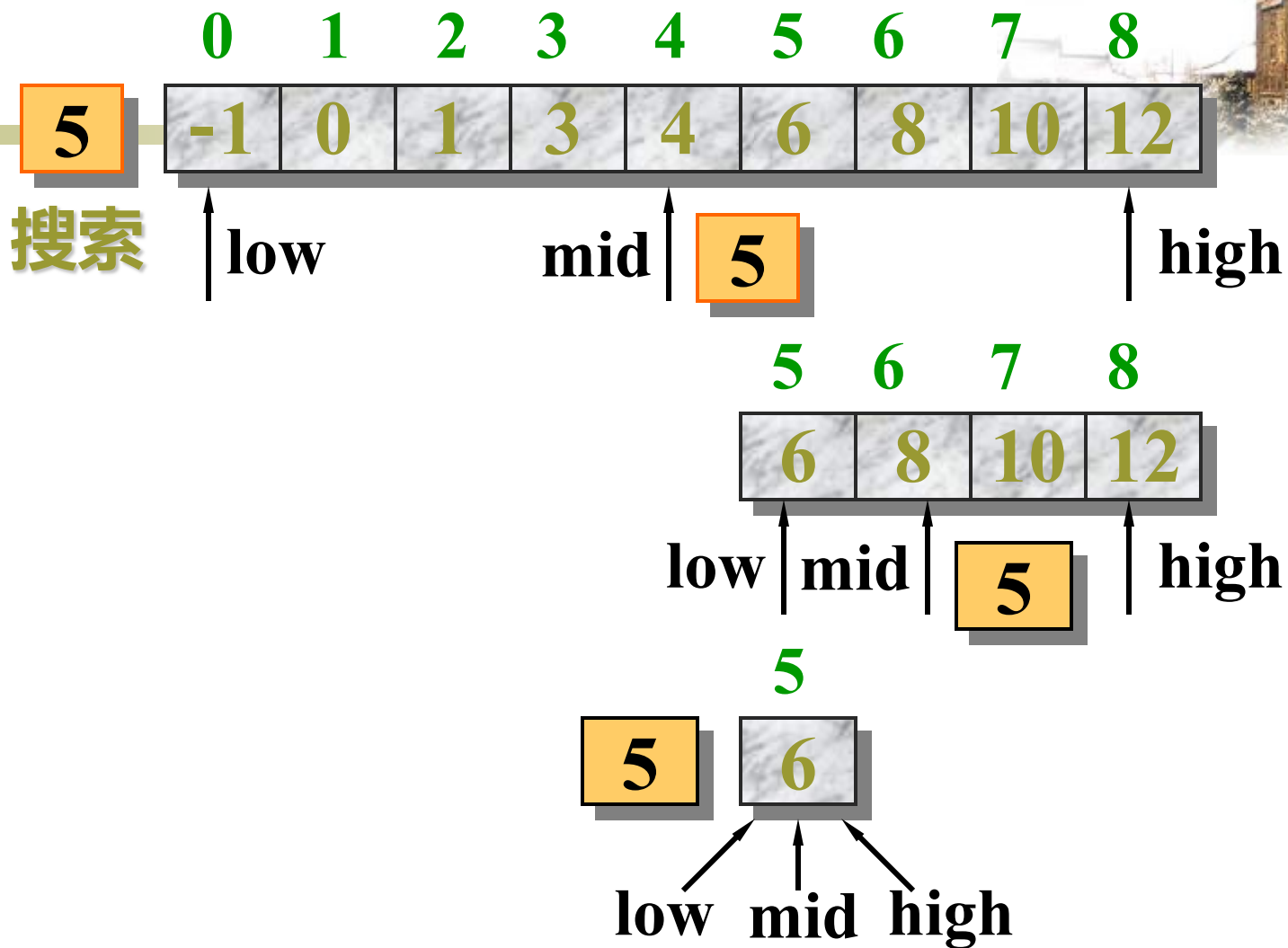
7.1.3 基于有序顺序表的折半搜索



- 前提条件：查找表必须是有序的顺序表
- 基本思想：每次比较中间元素，根据比较结果缩小一半搜索范围
- 设 n 个对象存放在一个有序顺序表中，并按其关键码从小到大排好了序。
- 折半搜索时，先求位于搜索区间正中的对象的下标 mid ，用其关键码与给定值 x 比较：
 - ◆ $Element[mid].key == x$ ，搜索成功；
 - ◆ $Element[mid].key > x$ ，把搜索区间缩小到表的前半部分，继续折半搜索；
 - ◆ $Element[mid].key < x$ ，把搜索区间缩小到表的后半部分，继续折半搜索。
- 如果搜索区间已缩小到一个对象，仍未找到想要搜索的对象，则搜索失败。



搜索成功的例子



搜索失败的例子



```
template <class K> int orderedList<K> ::  
BinarySearch ( const K & x, const int low,  
               const int high ) const {  
//折半搜索的递归算法  
    int mid = -1;  
    if ( low <= high ) {  
        mid = ( low + high ) / 2;  
        if ( Element[mid].key < x )  
            mid = BinarySearch ( x, mid + 1, high );  
        else if ( Element[mid].key > x )  
            mid = BinarySearch ( x, low, mid - 1 );  
    }  
    return mid;  
}
```




有序顺序表的折半搜索的判定树

(10, 20, 30, 40, 50, 60)

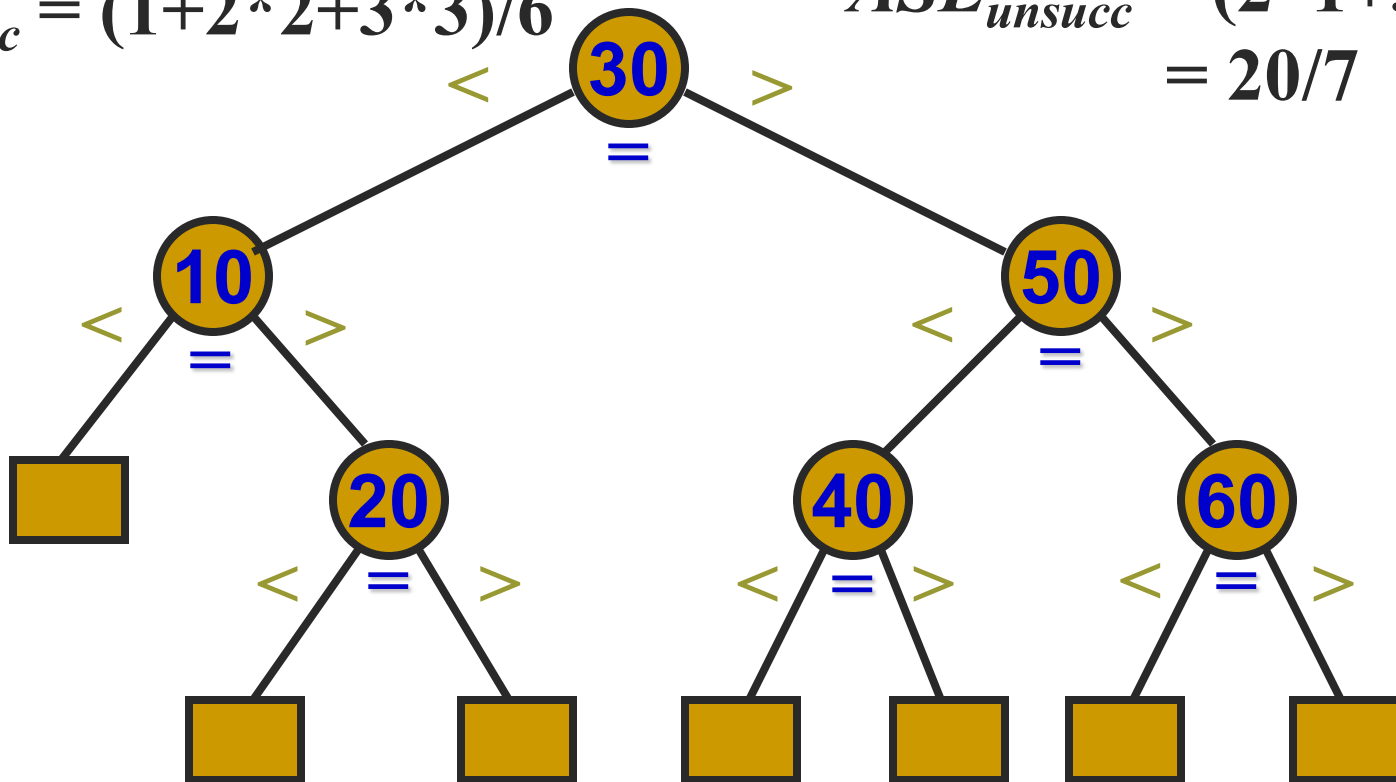


看圆

$$ASL_{succ} = (1 + 2 * 2 + 3 * 3) / 6 \\ = 14/6$$

看方

$$ASL_{unsucc} = (2 * 1 + 3 * 6) / 7 \\ = 20/7$$





设 $n=2^h-1$ ，则 $h=\log_2(n+1)$

$$\begin{aligned} \text{ASL}_{\text{succ}} &= \sum_{i=1}^n P_i \cdot Ci = \frac{1}{n} (1 \times 1 + 2 \times 2^1 + \cdots + h \times 2^{h-1}) \\ &= \frac{1}{n} ((h-1) \times 2^h + 1) \\ &= \frac{1}{n} ((\log_2(n+1) - 1) \times (n+1) + 1) \\ &= \frac{n+1}{n} \log_2(n+1) - 1 \end{aligned}$$

因此，折半搜索的 ASL_{succ} 为 $O(\log_2 n)$



判定树： 描述折半查找过程的二叉树

- 每个节点表示一次比较
- 成功节点在内部节点，失败节点在外部节点

优点： 效率高

缺点： 要求有序表，插入删除困难



7.2 二叉搜索树 (Binary Search Tree)



- 动态搜索结构，支持高效查找、插入和删除操作。

- 7.2.1 定义

二叉搜索树或者是一棵空树，或者是具有下列性质的二叉树：

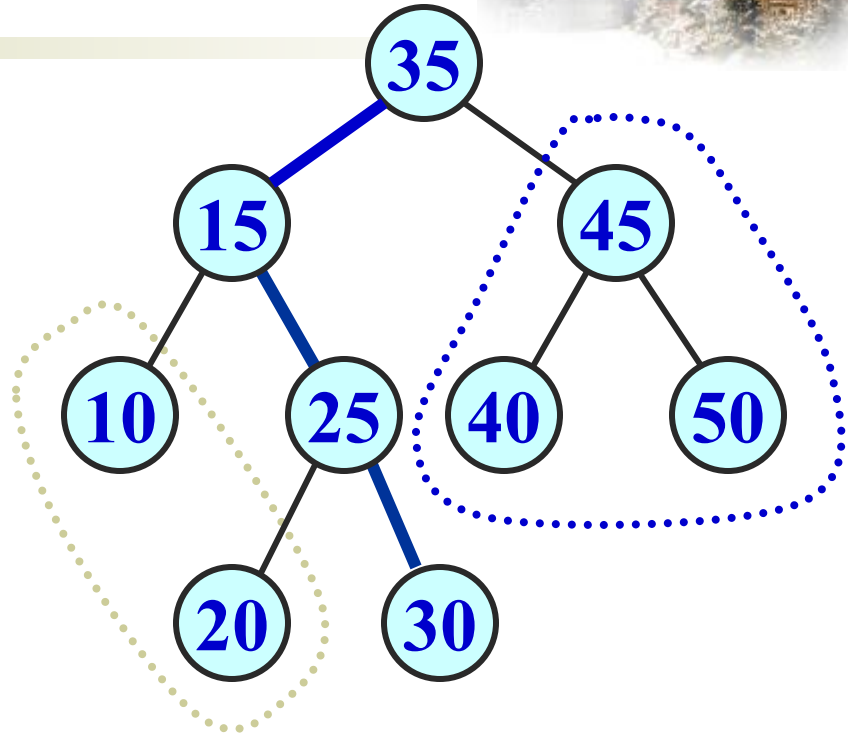
- ✓ 每个结点都有一个作为搜索依据的关键码 (key)，所有结点的关键码互不相同。
- ✓ 左子树（如果非空）上所有结点的关键码都小于根结点的关键码。
- ✓ 右子树（如果非空）上所有结点的关键码都大于根结点的关键码。
- ✓ 左子树和右子树也是**二叉搜索树**。



二叉搜索树示例



- 结点左子树上所有关键码小于该结点关键码；
- 右子树上所有关键码大于该结点关键码。



如果对一棵二叉搜索树进行中序遍历，可以按从小到大的顺序，将各结点关键码排列起来，所以也称二叉搜索树为二叉排序树。



二叉搜索树的类定义



```
#include <iostream.h>
#include <stdlib.h>
template <class K, class E>
struct BSTNode {                                //二叉树结点类
    K key;                                       // 关键码
    E data;                                     //除关键码外的其他数据域
    BSTNode<K, E> *left, *right; //左子女和右子女
    BSTNode() { left = NULL; right = NULL; } //构造函数
    BSTNode (const K k1, const E x, BSTNode<K, E> *L =
        NULL, BSTNode<K, E> *R = NULL) {
        key = k1; data = x; left = L; right = R;
    }                                           //构造函数
    ~BSTNode() {}                             //析构函数
```



```
void setData (K k1, E x) { key = k1; data = x; } //修改
E getData() { return data; } //提取
bool operator < (const BSTNode & x) //判小于
{ return key < x.key; }
bool operator > (const BSTNode & x) //判大于
{ return key > x.key; }
bool operator == (const BSTNode & x) //判等于
{ return key == x.key; }
};
```



```
template <class K, class E>
class BST {                                //二叉搜索树类定义
public:
    BST() { root = NULL; }                //构造函数
    BST(K value);                          //构造函数
    ~BST() {};                            //析构函数
    bool Search (const K x) const          //搜索
        { return Search(x,root) != NULL; }
    BST<K, E>& operator = (const BST<K, E>& R);
                                                //重载：赋值
    void makeEmpty()                       //置空
        { makeEmpty (root); root = NULL; }
```




```
void PrintTree() const { PrintTree (root); } //输出
```

```
E Min() { return Min(root)->data; } //求最小
```

```
E Max() { return Max(root)->data; } //求最大
```

```
bool Insert (const E& e1) //插入新元素  
{ return Insert(e1, root);}
```

```
bool Remove (const K x) { return Remove(x, root);}  
//删除含x的结点
```



private:

```
BSTNode<K, E> *root;    //根指针
K RefValue;             //输入停止标志
BSTNode<K, E> * Search (const K x, BSTNode<K, E> *ptr);
                        //递归： 搜索
void makeEmpty (BSTNode<K, E> *& ptr);
                        //递归： 置空
void PrintTree (BSTNode<K, E> *ptr) const;
                        //递归： 打印
BSTNode<K, E> * Copy (const BSTNode<K, E> *ptr);
                        //递归： 复制
```



```
BSTNode<K, E>* Min (BSTNode<K, E>* ptr);
```

```
//递归：求最小
```

```
BSTNode<K, E>* Max (BSTNode<K, E>* ptr);
```

```
//递归：求最大
```

```
bool Insert (const K k1, const E& e1, BSTNode<K, E>* & ptr);
```

```
//递归：插入
```

```
bool Remove (const K x, BSTNode<K, E>* & ptr);
```

```
//递归：删除
```

```
};
```

- 二叉搜索树的类定义用二叉链表作为它的存储表示，许多操作的实现与二叉树类似。



7.2.2 二叉搜索树的搜索



- 在二叉搜索树上进行搜索，是一个从根结点开始，沿某一个分支逐层向下进行比较判等的过程。它是一个递归的过程。
- 假设想要在二叉搜索树中搜索关键码为 x 的元素，搜索过程从根结点开始。
- 如果根指针为NULL，则搜索不成功；否则用给定值 x 与根结点的关键码进行比较：
 - ✓ 若给定值等于根结点关键码，则搜索成功，返回搜索成功信息并报告搜索到的结点地址。



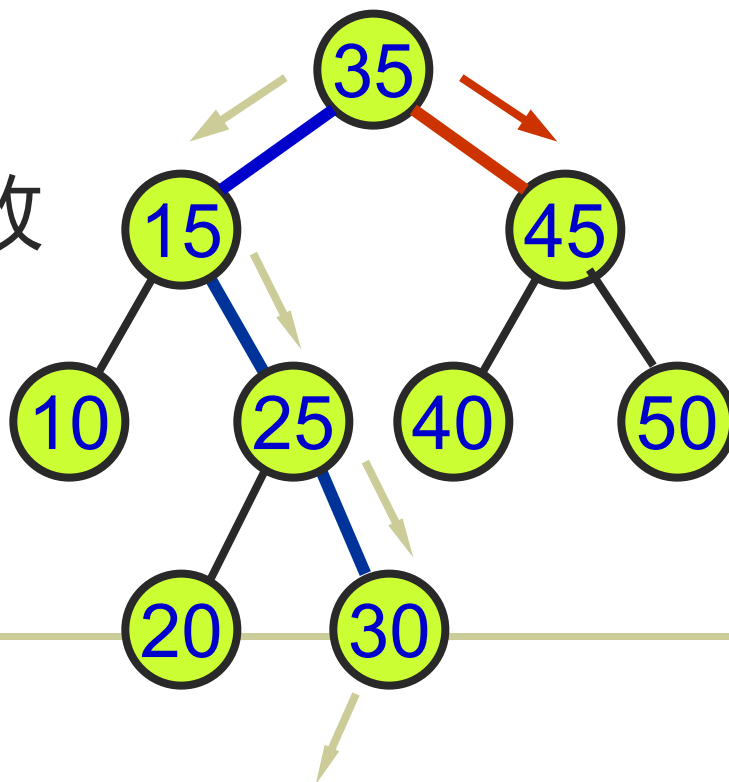
- ✓ 若给定值小于根结点的关键码，则继续递归搜索根结点的左子树；
- ✓ 否则递归搜索根结点的右子树。

搜索28

搜索失败

搜索45

搜索成功





```
template<class K, class E>
```

```
BSTNode<K, E>* BST<K, E>::
```

```
Search (const K x, BSTNode<K, E> *ptr) {
```

```
//私有递归函数：在以ptr为根的二叉搜索树中搜
```

```
//索含x的结点。若找到，则函数返回该结点的地址，
```

```
//否则函数返回NULL值。
```

```
    if (ptr == NULL) return NULL;
```

```
    else if (x < ptr->key) return Search(x, ptr->left);
```

```
    else if (x > ptr->key) return Search(x, ptr->right);
```

```
    else return ptr; //搜索成功
```

```
};
```



```
template<class K, class E>
```

```
BSTNode<K, E>* BST<K, E>::
```

```
Search (const K x, BSTNode<K, E> *ptr) {
```

```
//非递归函数：作为对比，在当前以ptr为根的二叉搜索树中  
//搜索含x的结点。若找到，则函数返回该结点的地址，否则  
//函数返回NULL值。
```

```
if (ptr == NULL) return NULL;
```

```
BSTNode<K, E>* temp = ptr;
```

```
while (temp != NULL) {
```

```
    if (x == temp->key) return temp;
```

```
    if (x < temp->key) temp = temp->left;
```

```
    else temp = temp->right;
```

```
}
```

```
return NULL;
```

```
};
```



- 搜索过程是从根结点开始，沿某条路径自上而下逐层比较判等的过程。
- 搜索成功，搜索指针将停留在树上某个结点；搜索不成功，搜索指针将走到树上某个结点的空子树。
- 设树的高度为 h ，最多比较次数不超过 h 。



7.2.3 二叉搜索树的插入



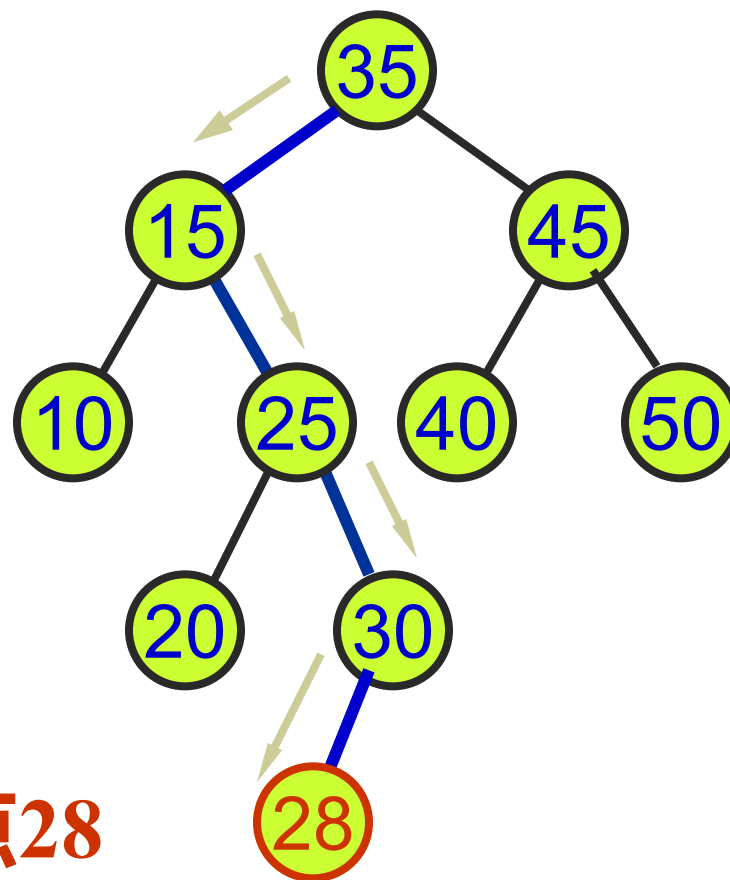
- 为了向二叉搜索树中插入一个新元素，必须先检查这个元素是否在树中已经存在。
- 在插入之前，先使用搜索算法在树中检查要插入元素有还是没有。
 - 如果**搜索成功**，说明树中已经有这个元素，不再插入；
 - 如果**搜索不成功**，说明树中原来没有关键码等于给定值的结点，把新元素加到搜索操作停止的地方。



7.2.3 二叉搜索树的插入



- 每次结点的插入，都要从根结点出发搜索插入位置，然后把新结点作为叶结点插入。



插入新结点28



二叉搜索树的插入算法



```
template <class K, class E>
bool BST<K, E>::Insert (const K k1, const E& e1,
    BSTNode<K, E> *& ptr) {
//私有函数：在以ptr为根的二叉搜索树中插入值为
//<k1,e1>的结点。若在树中已有含<k1,e1>的结点，则不插入
    if (ptr == NULL) {                                //新结点作为叶结点插入
        ptr = new BSTNode<K, E>(k1,e1);              //创建新结点
    }
    if (ptr == NULL)
        { cerr << "Out of space" << endl; exit(1); }
    return true;
}
```

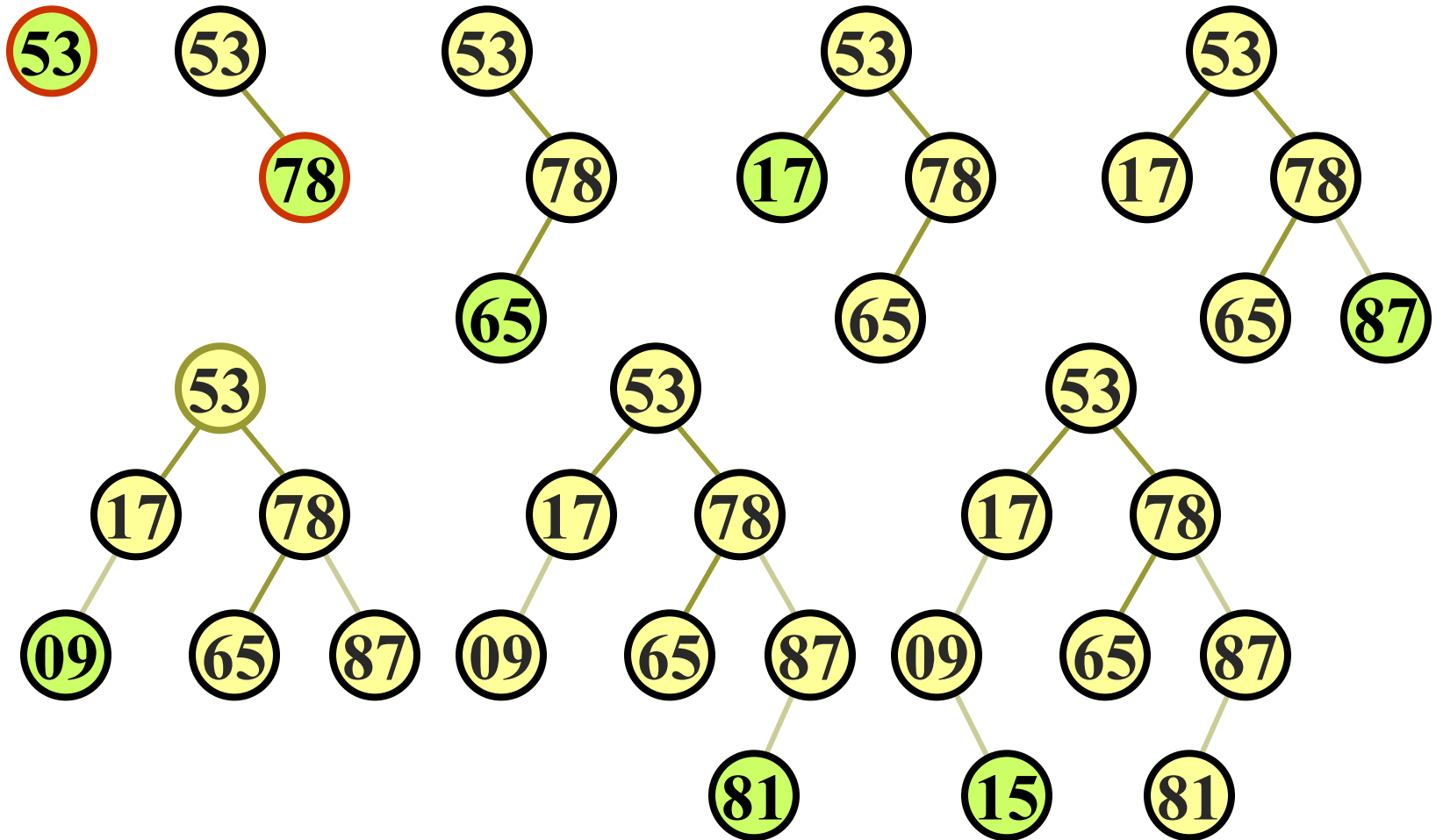


```
else if (k1 < ptr->key) Insert (k1,e1, ptr->left);  
    //左子树插入  
else if (k1 > ptr->key) Insert (k1,e1, ptr->right);  
    //右子树插入  
else return false;      //已在树中，不再插入  
};
```

- 利用二叉搜索树的插入算法，可以很方便地建立二叉搜索树。



输入数据 { 53, 78, 65, 17, 87, 09, 81, 15 }

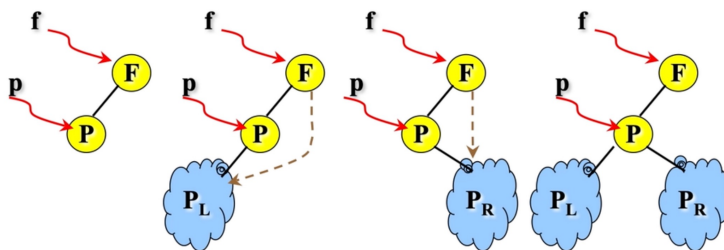




7.2.4 二叉搜索树的删除



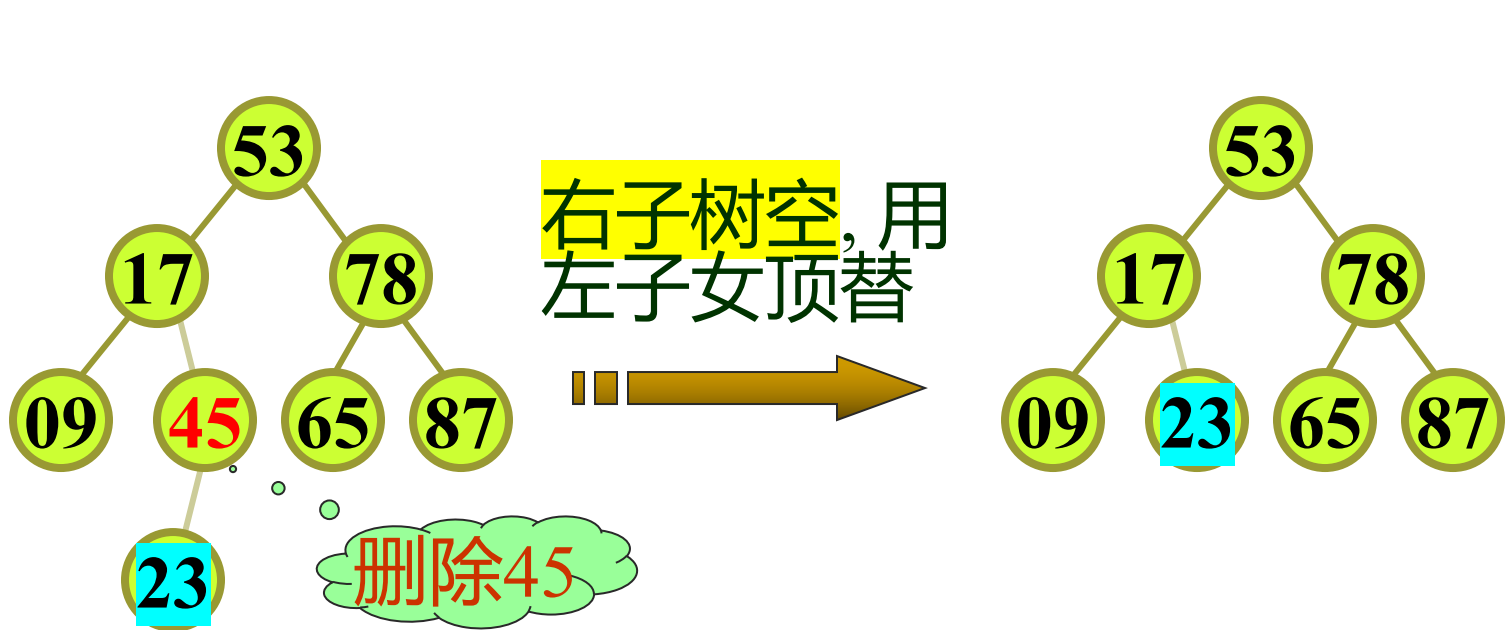
- 在二叉搜索树中删除一个结点时，必须将因删除结点而断开的二叉链表重新链接起来，同时确保二叉搜索树的性质不会失去。
- 为保证在删除后树的搜索性能不至于降低，还需要防止重新链接后树的高度增加。



- ✓ 删除叶结点，只需将其双亲结点指向它的指针清零，再释放它即可。

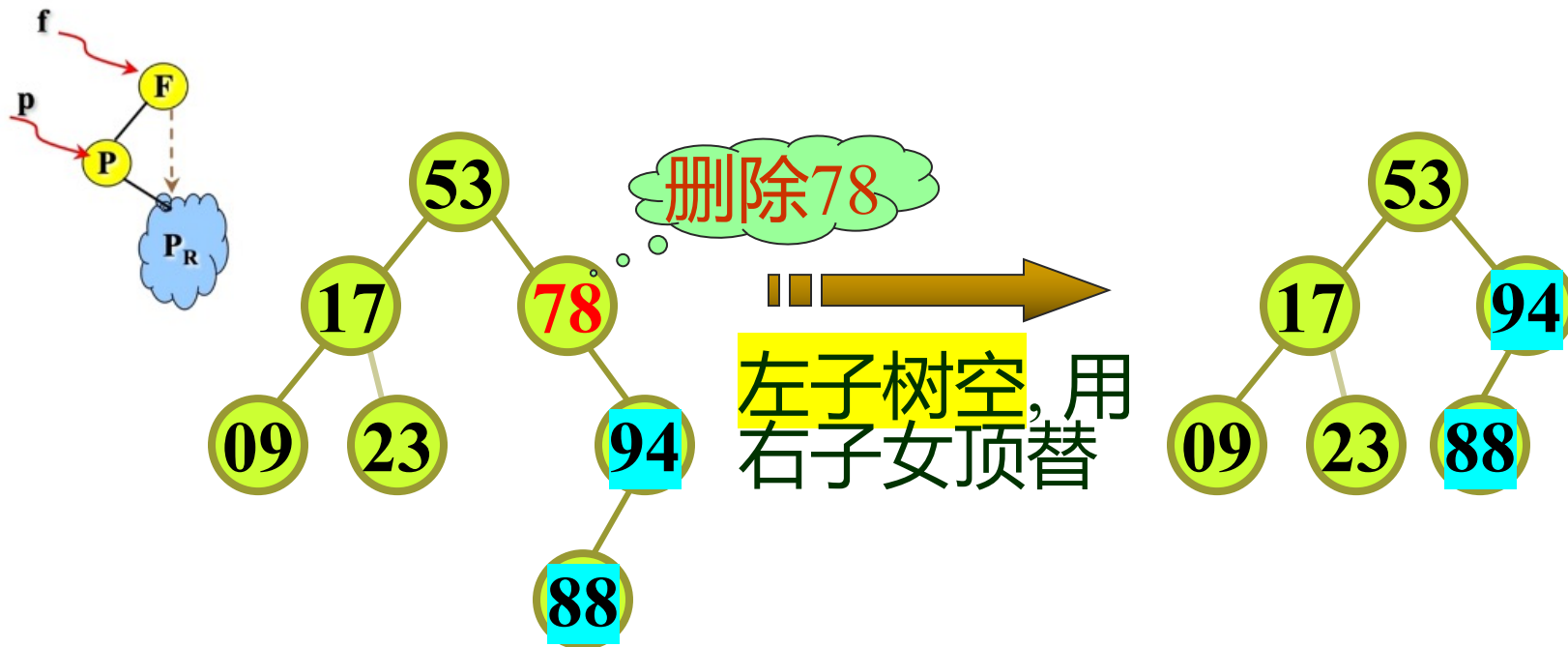


✓ 被删结点右子树为空，可以拿它的左子女结点顶替它的位置，再释放它。



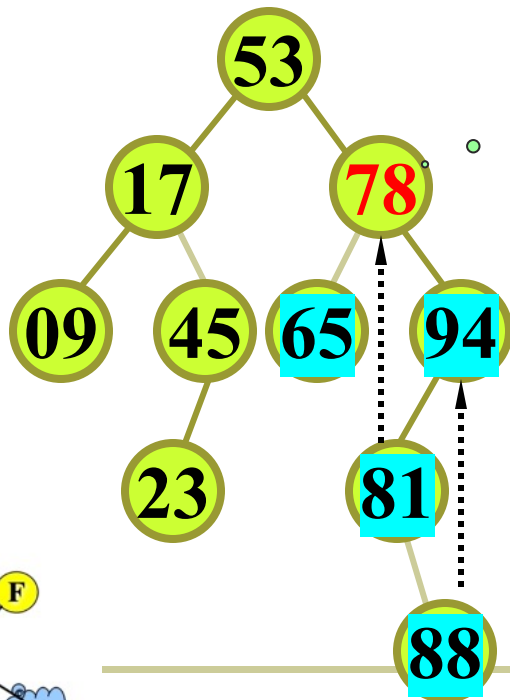


✓ 被删结点左子树为空，可以拿它的右子女结点顶替它的位置，再释放它。



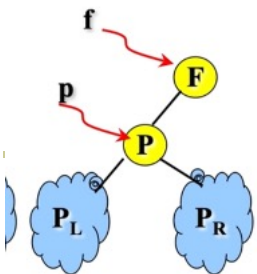
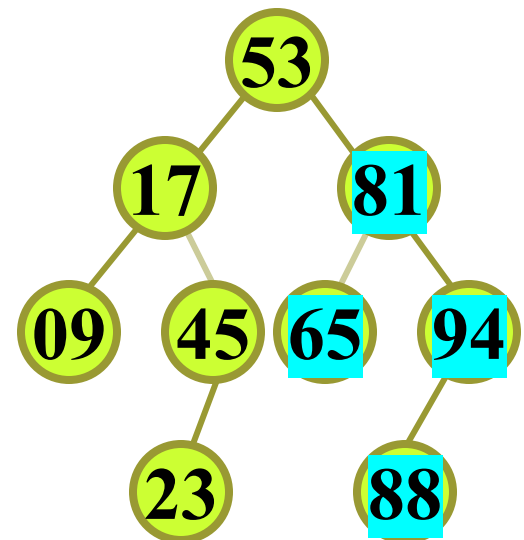


✓ 被删结点左、右子树都不为空，可以在它的右子树中寻找中序下的第一个结点(关键码最小)，用它的值填补到被删结点中，**再来处理这个结点的删除问题。**



删除78

左右子树不空，
在右子树上找
中序下第一个
结点填补





二叉搜索树的删除算法

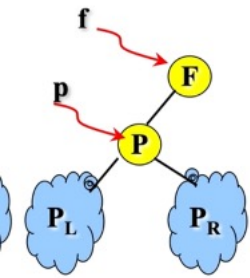


```
template <class K, class E>
bool BST<K, E>::Remove (const K x,
    BSTNode<K, E> *& ptr) {
    //在以 ptr 为根的二叉搜索树中删除含 x 的结点
    BSTNode<K, E> *temp;
    if (ptr != NULL) {
        if (x < ptr->key) Remove (x, ptr->left);
            //在左子树中执行删除
        else if (x > ptr->key) Remove (x, ptr->right);
            //在右子树中执行删除
    }
```



```
else if (ptr->left != NULL && ptr->right != NULL)
{
    //ptr指示关键码为x的结点，它有两个子女
    temp = ptr->right;
    //到右子树搜寻中序下第一个结点
    while (temp->left != NULL)
        temp = temp->left;
    ptr->key = temp->key;
    ptr->data = temp->data;
    //用该结点数据代替根结点数据
    Remove (ptr->key, ptr->right);
}
```

```
else { //ptr指示关键码为x的结点有一个子女
```





```
temp = ptr;
```

```
//ptr结点左子树为空
```

```
if (ptr->left == NULL) ptr = ptr->right;
```

```
//ptr结点右子树为空
```

```
else ptr = ptr->left;
```

```
delete temp;
```

```
return true;
```

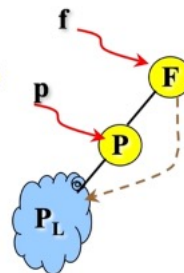
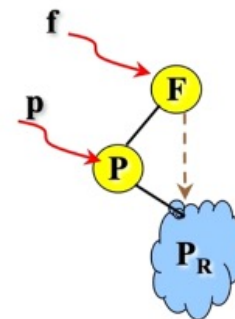
```
}
```

```
}
```

```
return false;
```

```
};
```

- 注意在删除算法参数表引用型指针参数的使用。





■ 优点:

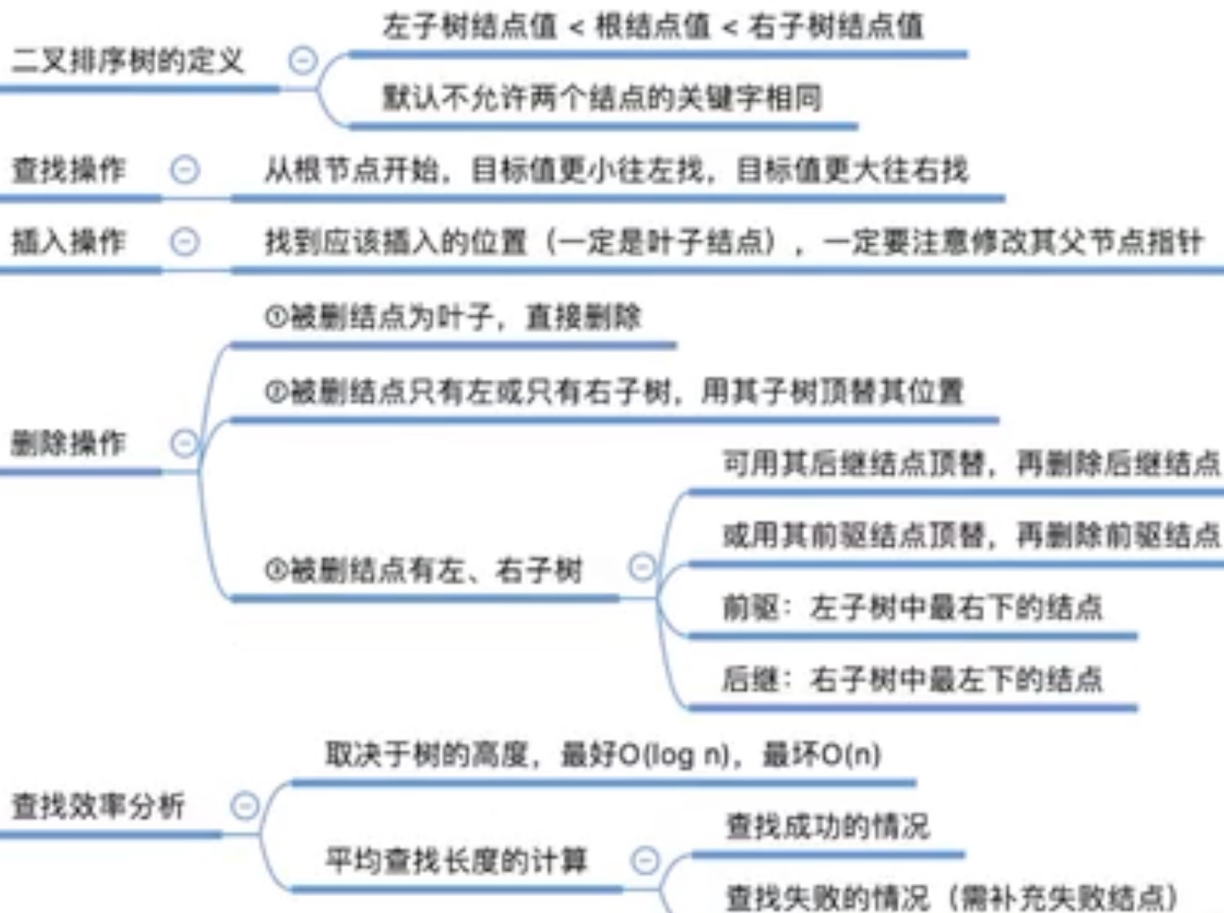
- 支持动态操作（插入、删除）
- 中序遍历可得有序序列
- 实现相对简单

■ 缺点:

- 可能退化成链表，性能下降
- 需要平衡机制（**AVL**树、红黑树等）



二叉排序树





特性	静态搜索结构	二叉搜索树
数据变化	基本不变	频繁变化
存储要求	有序（折半查找）	无特殊要求
查找效率	$O(\log n)$ （折半）	$O(h)$ （ h 为树高）
插入删除	困难，需要移动元素	相对容易
实现复杂度	简单	中等
内存使用	较少	需要指针开销
适用场景	静态数据集，查询频繁	动态数据集，需要频繁更新

1. 二叉搜索树的中序遍历序列是有序的
2. 相同关键字集合的不同插入顺序可能产生不同形状的二叉搜索树
3. n 个节点的二叉搜索树，最小高度为 $\lfloor \log_2 n \rfloor$ ，最大高度为 $n-1$
4. 静态查找适合数据稳定的场景，动态查找适合数据频繁变化的场景



- 有一个有序表：{2, 5, 8, 12, 16, 23, 38, 45, 56, 72, 91}
- 画出对该表进行折半查找的判定树
- 计算查找成功时的平均查找长度
- 计算查找38需要多少次比较

